Tejaswi Khode

Experiment 7

Aim

Perform chunking for the given text input.

Theory

Chunking, also known as shallow parsing, is the process of grouping individual tokens (words) into higher-level phrases based on their POS tags, without building a full parse tree. It works in two main steps:

1. **POS Tagging** – Each word in the sentence is tagged with its part of speech (noun, verb, adjective, etc.).
2. **Chunk Grammar Definition** – A grammar (using regular expressions over POS tags) is defined to identify patterns, e.g., a Noun Phrase (NP) chunk rule: `NP: {<DT>?<JJ>*<NN>}` This means an NP chunk consists of an optional determiner (DT), followed by zero or more adjectives (JJ), followed by a noun (NN).

Chunking uses a `RegexpParser` to apply these grammar rules and extract chunks from POS-tagged text, producing a shallow tree structure.

Application

- Named Entity Recognition (NER) as a preprocessing step
- Information Extraction systems
- Question Answering systems

Advantages

- Faster and computationally cheaper than full parsing
- Easier to implement and interpret than a complete syntactic parse tree
- Useful for extracting specific phrase-level information without needing complete grammatical structure

Limitations

- Cannot capture deep grammatical relationships between phrases (unlike full parsing)
- Accuracy heavily depends on the correctness of POS tagging
- Rule-based grammars may not generalize well to complex or ambiguous sentences

In [28]:

```
import pandas as pd
import nltk
import re
from nltk.tokenize import word_tokenize
from nltk import pos_tag
from nltk import RegexpParser
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
```

```
[nltk_data] Downloading package punkt to /home/computer/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /home/computer/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
```

Out[28]:

```
True
```

In [29]:

```
df = pd.read_csv("UpdatedResumeDataSet.csv")
print("Shape :",df.shape)
df.head()
```

```
Shape : (962, 2)
```

Out[29]:

	Category	Resume
0	Data Science	Skills * Programming Languages: Python (pandas...
1	Data Science	Education Details \r\nMay 2013 to May 2017 B.E...
2	Data Science	Areas of Interest Deep Learning, Control Syste...
3	Data Science	Skills â¯¢ R â¯¢ Python â¯¢ SAP HANA â¯¢ Table...
4	Data Science	Education Details \r\n MCA YMCAUST, Faridab...

In [30]:
```
print(df.columns)
df[['Category','Resume']].head()
```

```
Index(['Category', 'Resume'], dtype='object')
```

Out[30]:

	Category	Resume
0	Data Science	Skills * Programming Languages: Python (pandas...
1	Data Science	Education Details \r\nMay 2013 to May 2017 B.E...
2	Data Science	Areas of Interest Deep Learning, Control Syste...
3	Data Science	Skills â¯¢ R â¯¢ Python â¯¢ SAP HANA â¯¢ Table...
4	Data Science	Education Details \r\n MCA YMCAUST, Faridab...

In [17]:
```
sample_resume = df['Resume'].iloc[0]
# Keep only the first sentence (or first 20 words)
sample_resume = " ".join(sample_resume.split()[:20])
print(sample_resume)
```

```
Skills * Programming Languages: Python (pandas, numpy, scipy, scikit-learn, matplotlib), Sql, Java, JavaS
cript/JQuery. * Machine learning: Regression, SVM, NaÃ¯ve Bayes,
```

In [31]:
```
import re
sample_resume = sample_resume.lower()
sample_resume = re.sub(r'[^a-z\s]', ' ', sample_resume)
sample_resume = re.sub(r'\s+', ' ', sample_resume)
print(sample_resume)
```

```
skills programming languages python pandas numpy scipy scikit learn matplotlib sql java javascript jquery
machine learning regression svm na ve bayes
```

In [32]:
```
from nltk.tokenize import word_tokenize
tokens = word_tokenize(sample_resume)
print(tokens)
```

```
['skills', 'programming', 'languages', 'python', 'pandas', 'numpy', 'scipy', 'scikit', 'learn', 'matplotl
ib', 'sql', 'java', 'javascript', 'jquery', 'machine', 'learning', 'regression', 'svm', 'na', 've', 'baye
s']
```

In [33]:
```
from nltk import pos_tag
tagged = pos_tag(tokens)
print(tagged)
```

```
[('skills', 'NNS'), ('programming', 'VBG'), ('languages', 'NNS'), ('python', 'VBP'), ('pandas', 'JJ'),
('numpy', 'JJ'), ('scipy', 'NN'), ('scikit', 'NN'), ('learn', 'VBP'), ('matplotlib', 'NN'), ('sql', 'N
N'), ('java', 'NN'), ('javascript', 'NN'), ('jquery', 'NN'), ('machine', 'NN'), ('learning', 'VBG'), ('re
gression', 'NN'), ('svm', 'NN'), ('na', 'TO'), ('ve', 'VB'), ('bayes', 'NNS')]
```

In [34]:
```
from nltk import RegexpParser
grammar = r"""
NP: {<JJ>*<NN.*>+}
"""
chunk_parser = RegexpParser(grammar)
```

In [35]:
```
chunk_tree = chunk_parser.parse(tagged)
print(chunk_tree)
```

```
(S
  (NP skills/NNS)
  programming/VBG
  (NP languages/NNS)
  python/VBP
  (NP pandas/JJ numpy/JJ scipy/NN scikit/NN)
  learn/VBP
  (NP
    matplotlib/NN
    sql/NN
    java/NN
    javascript/NN
    jquery/NN
    machine/NN)
  learning/VBG
  (NP regression/NN svm/NN)
  na/TO
  ve/VB
  (NP bayes/NNS))
```

In [23]:
```
chunk_tree.pretty_print()
```

```
S
        _________________________________________________________________________________________________
____|____________________________________________________________________________________________________
________
       |            |          |          |         |     |       NP           NP                   NP
NP                                                     NP            NP
       |            |          |          |         |     |       |            |            ________|____
______________             _________________|_________________________________             ________|____
|
programming/VBG python/VBP learn/VBP learning/VBG na/TO ve/VB skills/NNS languages/NNS pandas/JJ numpy/JJ
scipy/NN scikit/NN matplotlib/NN sql/NN java/NN javascript/NN jquery/NN machine/NN regression/NN     svm/
NN bayes/NNS
```

In [36]:
```
print("Extracted Noun Phrases:\n")
for subtree in chunk_tree.subtrees():
    if subtree.label() == "NP":
        print(" ".join(word for word, tag in subtree.leaves()))
```

```
Extracted Noun Phrases:

skills
languages
pandas numpy scipy scikit
matplotlib sql java javascript jquery machine
regression svm
bayes
```

In [37]:
```
text = "Experienced Python developer with machine learning and data analysis skills."
tokens = word_tokenize(text)
tagged = pos_tag(tokens)
grammar = r"""
NP: {<JJ>*<NN.*>+}
"""
chunk_parser = RegexpParser(grammar)
chunk_tree = chunk_parser.parse(tagged)
chunk_tree.pretty_print()
```

```
                                                   S
    _______________________________________________|________________________________________________
   |      |     |                      NP                              NP                           NP
   |      |     |          ____________|____________            _______|_______           __________|____
______
with/IN and/CC ./. Experienced/NNP Python/NNP developer/NN machine/NN     learning/NN data/NNS analysis/N
N skills/NNS
```